



[PROJECT NO. CS-101]

GIT & GITHUB

VERSION CONTROL MADE EASY

A BEGINNER'S GUIDE FOR STUDENTS

WHAT IS GIT?



A VERSION CONTROL SYSTEM (VCS)

Tracks every change in your code over time



SAVES SNAPSHOTS (COMMITTS)

Like checkpoints in a video game



WORKS OFFLINE

You don't need the internet to use it



FAST & FREE & OPEN SOURCE

Used by millions of developers worldwide

ANALOGY DIAGRAM

GIT = TIME MACHINE FOR CODE

- v1.0 → Initial code
- v1.1 → Fixed bug
- v1.2 → Added feature
- v2.0 → Major update

← YOU CAN GO BACK ANYTIME

WHY **VERSION** CONTROL?



WITHOUT GIT:

OVERWRITING EACH OTHER

✓ Git tracks who changed what



WITHOUT GIT:

USB / EMAIL SHARING

✓ Everyone has the full history



WITHOUT GIT:

LOSING WORK (NO BACKUP)

✓ Every commit is a restore point



WITHOUT GIT:

"WHICH VERSION IS FINAL?"

✓ One source of truth always

KEY GIT CONCEPTS



REPOSITORY

REPO

A folder tracked by Git containing your project and all its history

ANALOGY: *Like a project binder with every draft inside*



COMMIT

SNAPSHOT

A saved checkpoint of your files at a specific point in time

ANALOGY: *Like hitting "Save" in a game – you can return to this*



BRANCH

COPY

A parallel version of your code to try new ideas safely

ANALOGY: *Like a rough draft – write freely without ruining the original*



MERGE

COMBINE

Combines changes from one branch into another

ANALOGY: *Like finalizing your rough draft into the main document*

BASIC GIT COMMANDS

`git init` → Initialize a new Git repository

`git status` → Show what files have changed

`git add .` → Stage all changes for commit

`git commit -m "msg"` → Save a snapshot with a message

`git push` → Upload commits to remote (GitHub)

`git pull` → Download latest changes from remote

EXAMPLE SESSION

```
bash
$ git init
  Initialized empty Git repository
$ git add .
$ git status
  Changes to be committed:
    new file: index.html
$ git commit -m "first commit"
[main] first commit
$ git push origin main
  Branch pushed to GitHub ✓
```

[SECTION 06]

WHAT IS GITHUB?

GitHub is a **cloud platform** that hosts Git repositories and adds powerful collaboration tools on top.

ANALOGY

Git is like **Microsoft Word** (the tool).
GitHub is like **Google Drive** (where you store and share it).

KEY FEATURES



CLOUD STORAGE

Your Git repo hosted online – accessible anywhere



COLLABORATION

Multiple people can work on the same project



CODE REVIEW

Review and discuss changes before merging



ISSUE TRACKING

Track bugs and feature requests in one place



OPEN SOURCE

Millions of public projects you can learn from



CI / CD

Automatically test and deploy your code



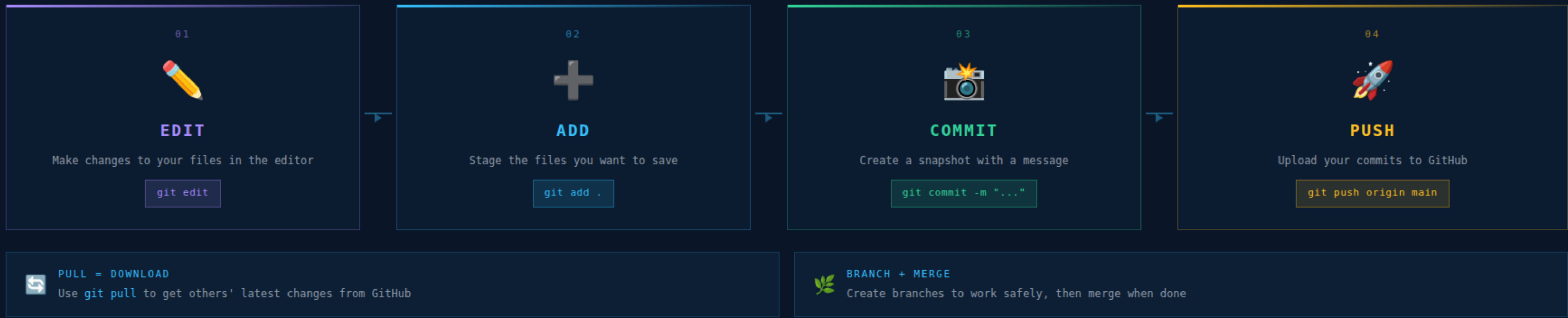
Over **100 million** developers use GitHub worldwide

GIT VS GITHUB

ASPECT	GIT	GITHUB
TYPE	Local tool on your computer	Online cloud platform
PURPOSE	Track & manage code versions	Host, share & collaborate
INTERNET	Works offline	Requires internet connection
INTERFACE	Command line (terminal)	Web browser + GUI
STORAGE	On your hard drive	On GitHub's servers
SHARING	Not built-in	Core feature
FREE?	✓ Yes, always	✓ Free tier available

💡 Remember: You use Git locally, then push to GitHub to share with the world.

THE GIT WORKFLOW



REAL-WORLD USE CASES



TEAM PROJECTS

COLLABORATION

- ▶ Each member works on their own branch
- ▶ No one overwrites each other's code
- ▶ Merge reviews keep quality high



OPEN SOURCE

COMMUNITY

- ▶ Fork any public repo to experiment
- ▶ Submit pull requests with improvements
- ▶ Learn from millions of real codebases



PORTFOLIOS

CAREER

- ▶ Showcase your projects publicly
- ▶ Prove your coding activity with commits
- ▶ Employers check GitHub profiles



INDUSTRY

PROFESSIONAL

- ▶ Every tech company uses Git
- ▶ CI/CD pipelines trigger on push
- ▶ Code review is standard practice

[SECTION 10]

TIPS & CONCLUSION

Every professional developer uses Git daily. The best way to learn is to **start a project today** and commit as you go.

YOUR FIRST STEPS

- 1 Install Git → git-scm.com
- 2 Create a GitHub account
- 3 Make your first repository
- 4 Push your first commit

BEST PRACTICES



WRITE CLEAR COMMIT MESSAGES

"Fixed login bug" not "update stuff"



USE BRANCHES FOR NEW FEATURES

Never code experiments on main



PULL BEFORE YOU PUSH

Always sync before uploading changes



ADD A README TO EVERY PROJECT

Explain what your project does



COMMIT OFTEN, PUSH REGULARLY

Small commits are easier to understand

YOU'VE GOT THIS 🚀

The only way to learn Git is by doing. Start small, stay consistent.